

HealthAI Coach

Backend Data Engineering

Nutrition, activité physique, qualité des données — de l'ingestion à l'orchestration

Alexandre Laugier · Florian Abgrall · Johane Decamps · William Mibelli

Juillet 2026

Sommaire

1. Contexte & objectifs
2. Architecture du système
3. Modélisation des données
4. Pipeline ETL & orchestration Airflow
5. API REST
6. Interface admin & qualité des données
7. Dockerisation
8. Tests & CI/CD
9. Démonstration live
10. Bilan & conclusion

01 · Contexte & objectifs

HealthAI Coach, c'est quoi ?

- Plateforme de **coaching santé** : suivi nutritionnel, activité physique, relevés biométriques, profil médical déclaratif
- Génération de recommandations personnalisées (diététiques aujourd'hui, plans d'entraînement/nutrition via microservice IA à venir)
- Ce dépôt = le **backend data** uniquement : base de données, pipeline ETL, API REST, supervision qualité, conteneurisation
- Pas de frontend utilisateur final — hors périmètre de cette formation

Modèle économique

Deux offres, qui conditionnent des choix techniques concrets :

- **Freemium** — `free` / `premium` / `premium_plus` → fonctionnalités IA réservées aux paliers payants (gating côté API)
- **B2B en marque blanche** — organisations partenaires (salles de sport, mutuelles, entreprises) → provisionné par un admin, pas de self-service pour ce palier

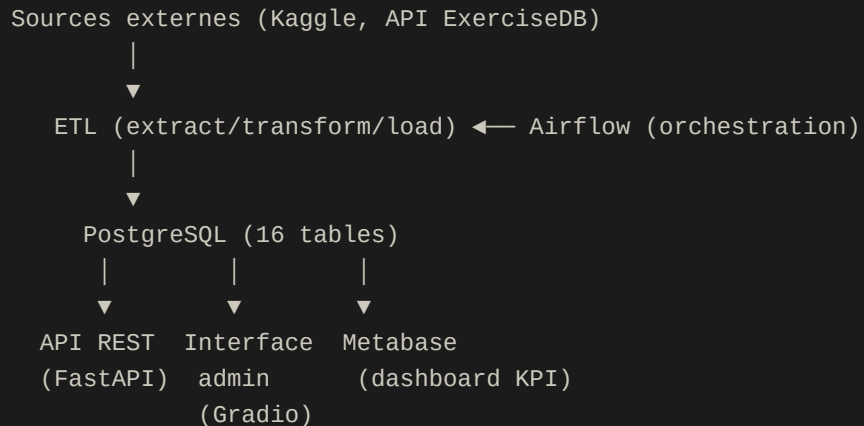
Répartition des rôles

Rôle	Périmètre	Qui
A	Architecture, base de données, interface admin, Metabase, dockerisation	Alexandre
B	API REST (FastAPI)	Florian Abgrall
C	Pipeline ETL, orchestration Airflow	Johane, William

Flux de travail : une étape = une Pull Request, tests obligatoires, revue avant fusion

02 · Architecture du système

Vue d'ensemble



Tout conteneurisé via `docker compose` (sept services), une seule commande

Choix technologiques

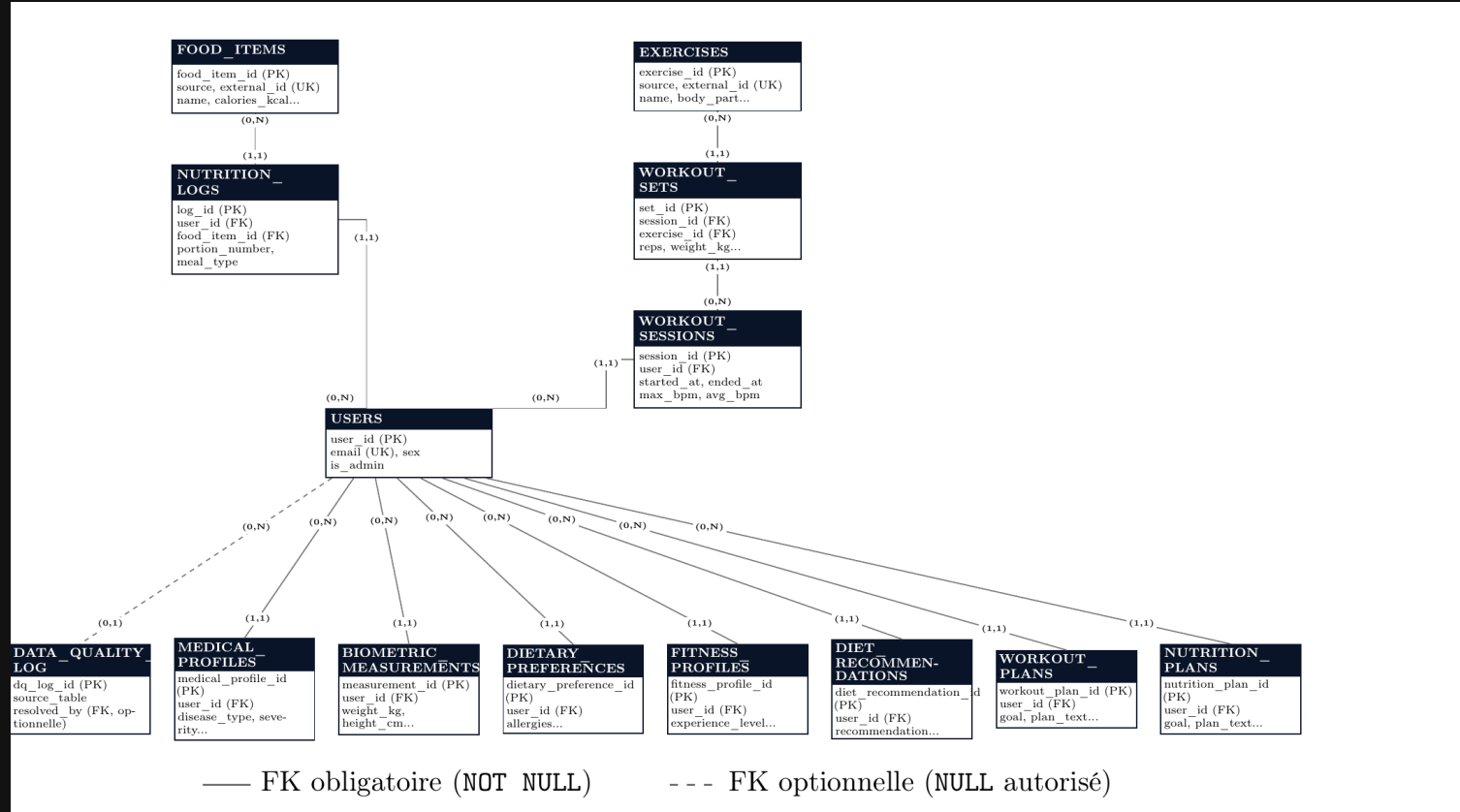
Composant	Techno	Pourquoi
Base de données	PostgreSQL	Domaine relationnel — contraintes CHECK/FK portent les règles métier
Migrations	Alembic	Schéma versionné et rejouable (upgrade/downgrade), historique traçable en équipe
Orchestration	Apache Airflow	
API	FastAPI	SQLAlchemy, Pydantic, JWT
Dashboard	Metabase	
Interface admin	Gradio	Outil interne à faible surface, rapidité prime sur personnalisation
Conteneurisation	Docker Compose	

03 · Modélisation des données

Le schéma en un coup d'œil

- **16 tables**, 5 domaines fonctionnels autour de `USERS` : nutrition · activité physique · suivi de santé déclaratif · abonnements · contenu généré par IA
- Schéma **auto-documenté** : `COMMENT ON TABLE/COLUMN` directement en base → les conventions non triviales survivent, pas seulement dans un doc externe qui peut diverger

Modèle Conceptuel de Données



Conventions d'intégrité

- **Idempotence au chargement** : `UNIQUE(source, external_id)` sur `food_items / exercises` → `upsert ( ON CONFLICT DO UPDATE )`, les DAG Airflow peuvent être rejoués sans dupliquer
- **Génération de données manquantes** : `Faker.seed()` déterministe par ligne source → un même utilisateur ne change pas de nom à chaque ré-exécution du pipeline

Un bug réel, capturé par la contrainte plutôt que par les tests :

```
sex VARCHAR(10) CHECK (sex IN ('F', 'M', 'other'))
```

```
# ETL : 'Other' (majuscule) ≠ 'other' → rejeté par PostgreSQL,  
# mais les 4 tests unitaires du module (entrées mockées) passaient  
users["sex"] = users["sex"].replace({"None": "Other"})
```

Conventions d'intégrité (suite)

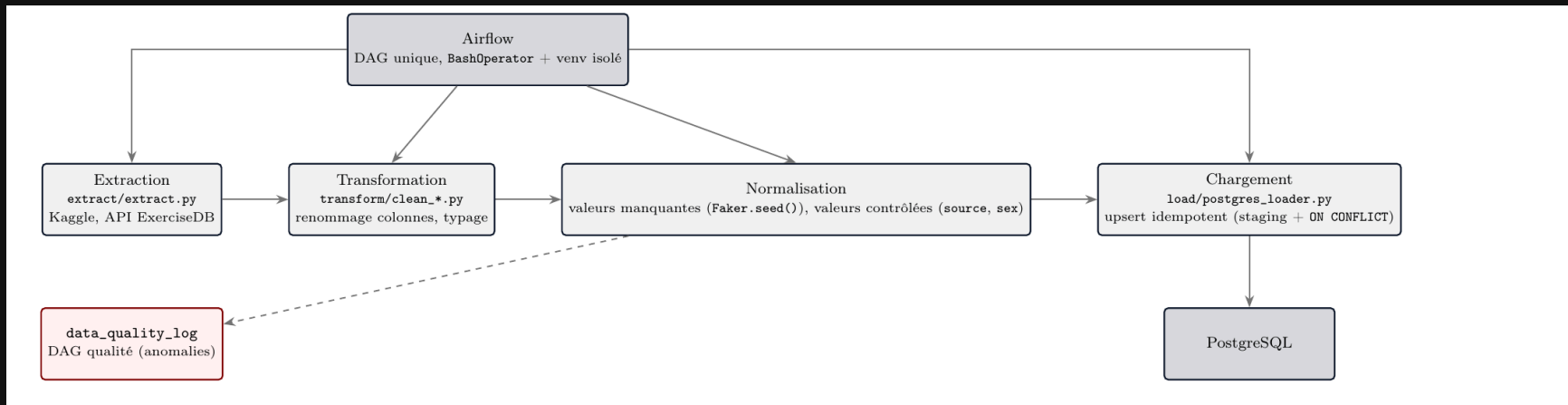
- **Cohérence métier en base**, pas dans le code applicatif :

```
CONSTRAINT chk_subscriptions_b2b_organization  
CHECK ((tier = 'b2b' AND organization_id IS NOT NULL)  
       OR (tier <> 'b2b' AND organization_id IS NULL))
```

- Garantit qu'une organisation n'est renseignée que pour les abonnements B2B, et inversement — impossible à contourner en écrivant directement en base, contrairement à une validation uniquement côté API

04 · Pipeline ETL & orchestration Airflow

Structure du pipeline



Extraction multi-sources (E)

- **ExerciseDB API** : pagination par curseur (`nextCursor`) pour récupérer l'intégralité du catalogue
- **Kaggle** : 4 datasets téléchargés automatiquement (Nutrition, Diet, User, Activity)
- **Rate limiting** : décorateurs `@limits` / `@sleep_and_retry` pour respecter les quotas d'API et éviter les blocages
- Stockage brut (CSV/JSON) avant toute transformation — étape rejouable indépendamment du reste du pipeline

Nettoyage & anonymisation (T)

- **Anonymisation** : `Faker` avec `seed=42` déterministe — faux noms/emails cohérents, mais reproductibles d'un run à l'autre (tests, debug)
- **Normalisation** : `'Male'/'Female'` → `'M'/'F'` , valeurs manquantes → `'other'`
- **Dérivation de champs** : date de naissance calculée depuis l'âge fourni, `ended_at` des sessions calculé depuis la durée

Chargement PostgreSQL (L)

- **Tables de staging** : injection rapide via `to_sql` dans des tables temporaires, avant bascule vers les tables finales
- **Stratégie UPSERT** : `ON CONFLICT ... DO UPDATE` pour les catalogues (`food_items` , `exercises`) — idempotent, pas de doublons en cas de relance
- **Historisation différenciée** : séries temporelles biométriques ajoutées en continu, dimensions (profils utilisateurs) synchronisées

Orchestration Airflow

- **Un DAG unique** plutôt qu'un DAG par source : le volume ne justifie pas plus de granularité
- `extract` → `transform_and_load` → `quality_check`, séquentiel, `@daily`
- **Problème réel** : Airflow exige `SQLAlchemy<2.0` en interne, incompatible avec le `2.0.51` utilisé partout ailleurs → **venv Python isolé** (`/opt/etl-venv`), tâches en `BashOperator`

```
# airflow/dags/healthai_etl_pipeline.py
transform_and_load_task = BashOperator(
    task_id="transform_and_load",
    bash_command=f"cd {WORKDIR} && {ETL_PYTHON} -c "
                "'from etl.load.postgres_loader import load_data; load_data()'",
)
```

- Testé de bout en bout avec de **vraies données** : 2851 users, 645 food_items, 404 exercices chargés

05 · API REST

Structure

- FastAPI, toutes les routes préfixées `/api/v1`
- Architecture en couches : `routers/` (HTTP) → `crud/` (accès DB) → `models/` (ORM) + `schemas/` (Pydantic)
- Authentification **JWT** (OAuth2 password flow) : `POST /auth/login`
- Trois dépendances réutilisables : `CurrentUser` , `CurrentAdmin` , `CurrentPremiumUser`
- Documentation interactive auto-générée : `/docs` (Swagger), `/redoc`

Endpoints principaux

- **auth / users / admin** — inscription, connexion, profil, gestion admin
- **food-items / exercises** — catalogues ETL, lecture libre, écriture admin
- **nutrition-logs / workouts / biometrics / health-profiles** — données propres à l'utilisateur
- **data-quality** — vue admin sur les anomalies, alternative programmatique à l'interface Gradio
- **subscriptions / organizations** — abonnements self-service + provisionnement B2B
- **ai** — génération de contenu, réservée aux paliers payants

Abonnements & microservice IA

- Contenu IA (`/ai/diet-recommendations` , `/ai/workout-plans` , `/ai/nutrition-plans`) : 403 en `free` , 401 sans auth
- Une dépendance FastAPI réutilisable porte tout le contrôle d'accès :

```
# api/core/deps.py
def get_current_premium_user(current_user: CurrentUser, db: DB) -> User:
    subscription = get_active_subscription(db, current_user.id)
    if not subscription or subscription.tier not in PREMIUM_TIERS:
        raise HTTPException(status_code=403, detail="Premium subscription required")
    return current_user

CurrentPremiumUser = Annotated[User, Depends(get_current_premium_user)]
```

- Microservice pas encore déployé → **client stub** (`api/services/ai_client.py`) reproduisant la signature du futur appel HTTP réel — brancher le vrai microservice ne changera que ce fichier
- Vérifié par des tests réels contre PostgreSQL (gating, persistance, isolation par utilisateur)

06 · Interface admin & qualité des données

Interface admin (Gradio)

 **Qualité des données**

Consultation, correction manuelle et export des anomalies détectées par l'ETL.

Filtres

Sévérité

Statut de résolution

Table source (filtre partiel)

Rafraîchir

Toutes

Non résolues

ex. food_items

Anomalies détectées

dq_log_id	source_table	source_record_id	dag_id	rule_name	severity	message	detected_at	reso
1	food_items			check_negative_values	warning	Valeur négative détectée sur une co...	2026-07-09 12:11	fals
2	nutrition_logs			check_fk_integrity	error	food_item_id introuvable dans food_...	2026-07-09 12:11	fals
3	biometric_mesure_...			check_range	info	body_fat_pct hors de la plage 0-100...	2026-07-09 12:11	fals

Correction manuelle

Identifiant de l'anomalie (dq_log_id)

Résolu par (administrateur)

Marquer comme résolu

Numéro affiché dans la colonne 'dq_log_id' du tableau ci-dessus.

Optionnel : sélectionnez le compte admin qui traite cette anomalie, pour tracer qui a résolu quoi. Laissez sur 'Non renseigné' si vous ne souhaitez pas l'indiquer.

0

admin@healthai.local (#1)

Export

Exporter en CSV

Exporter en JSON

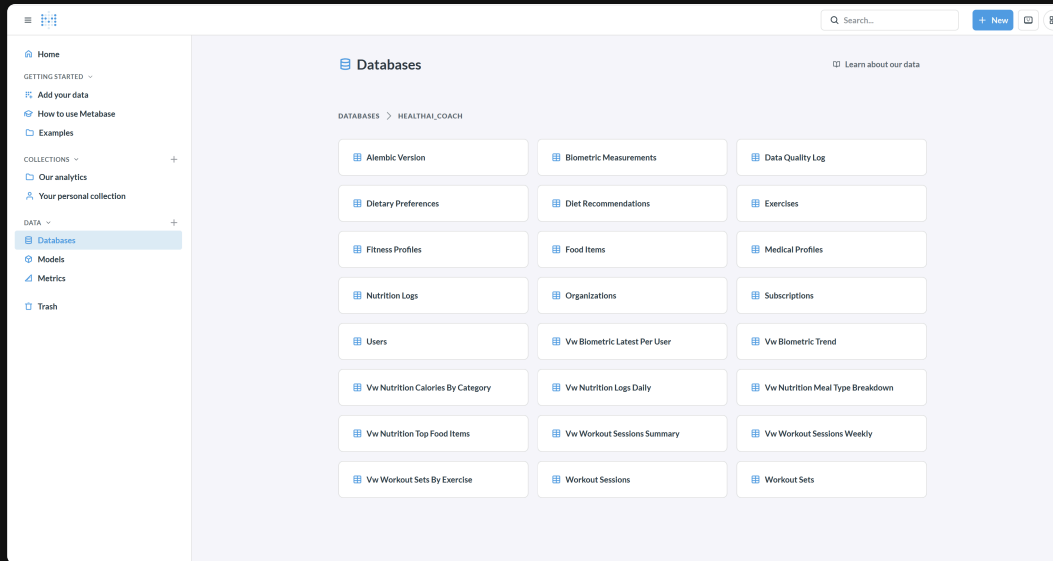
Fichier exporté

Accessibilité RGAA — démarche

- Audit **automatisé** (pas seulement une relecture visuelle) pour objectiver la conformité réelle
- **Corrigé** : contraste insuffisant (labels 4,34:1, texte d'aide 2,56:1, seuil requis 4,5:1) → couleurs foncées explicitement ; hiérarchie de titres invalide (h1 → h3 sans h2) → corrigée
- **Non corrigible depuis notre code** : le composant `Dataframe` de Gradio génère des violations ARIA internes (éléments imbriqués, noms accessibles manquants) — documenté comme limitation connue

07 · Dashboard Metabase

Metabase



- 14 tables + plusieurs vues KPI pré-construites (`vw_nutrition_meal_type_breakdown` , `vw_biometric_trend` ...)
- Limitation assumée : personnalisation visuelle restreinte au plan gratuit (pas de retrait du branding, palettes limitées)

08 · Dockerisation

Sept services, une commande

```
docker compose up -d --build
```

- **postgres** → **db-init** (migrations + seed + vues KPI) → **admin_interface** / **metabase** / **airflow-init** → **airflow-webserver** / **airflow-scheduler**
- Airflow en `LocalExecutor` + base partagée sur le postgres du projet, pas le stack Celery+Redis complet vu en formation → un DAG unique ne justifie pas plusieurs workers, et la RAM de la machine de démo ne suit pas

Point de vigilance — confirmé en conditions réelles

- `db-init` (démon) fait un `TRUNCATE` avant repeuplement ; le vrai pipeline ETL ne tronque jamais
- **Testé en pratique** : après avoir chargé de vraies données via le DAG Airflow, un simple redémarrage d' `admin_interface` a **redéclenché** `db-init` et tout effacé
- Pas un risque théorique — un incident reproductible, documenté, décision encore à trancher

09 · Tests & CI/CD

Suite de tests

Module	Périmètre	Fichiers
tests/data_quality/	Schéma SQL : FK, CHECK, cascades	1
tests/admin_interface/	Logique pure + DB réelle	3
tests/api/	Auth, users, organizations, subscriptions	7
tests/etl/	Transform (mockées) + quality (DB réelle)	3

141 tests, GitHub Actions sur chaque PR (migrations + suite complète contre Postgres éphémère)

10 · Démonstration live

Ce qu'on va montrer

1. **Interface admin** — consultation, résolution, export des anomalies
2. **Metabase** — schéma et KPI
3. **API** — Swagger, authentification, gating premium
4. **Airflow** — déclenchement du DAG en direct, vraies données remplaçant les données de démo

11 · Bilan & conclusion

Obstacles techniques rencontrés

- **Les tests mockés ne remplacent pas une vraie base** — un bug de valeur (`'other'` au lieu de `'other'`) passait les tests unitaires mais violait une contrainte réelle, détecté seulement contre Postgres
- **Ma vérification RGAA manuelle était incomplète** — l'audit automatisé a trouvé ce que j'avais raté
- **Reproduction fidèle de la CI** — un échec non reproductible en local car l'environnement de test avait plus de dépendances que la CI réelle
- **Conflit SQLAlchemy avec Airflow, `db-init` vs pipeline réel** — détectés et documentés avant incident

Points positifs

- Contraintes d'intégrité portées par PostgreSQL, pas le code applicatif — survivent à n'importe quel point d'entrée
- Schéma auto-documenté, conventions lisibles depuis la base elle-même
- Convention d'idempotence définie une fois, reprise à l'identique sur chaque nouvelle source
- Discipline de revue systématique : checkout isolé, tests rejoués contre Postgres frais, jamais de confiance aveugle

Conclusion

- Schéma PostgreSQL complet, API authentifiée, **pipeline ETL orchestré et testé en conditions réelles**, interface admin auditée RGAA, dashboard Metabase, dockerisation en une commande
- Facteur limitant : le **temps** de la formation, pas un blocage technique → microservice IA esquissé (stub), conflit `db-init` / Airflow documenté mais pas tranché
- Choix assumé : prioriser la **robustesse** de ce qui est livré plutôt que l'exhaustivité du périmètre imaginé au départ

Merci

Questions ?